

TéléPro CI

Rapport de mise en place de la suite de tests

Couverture complète API + Web · Avril 2026

Résultat global

Tests web (React)**38 / 38**

Tests API unitaires**17 / 17**

Tests API intégration**33 / 33**

TOTAL**88 / 88**

Généré le 19 avril 2026

Dans le cadre du développement de TéléPro CI (application de gestion commerciale pour distributeurs de téléphones en Côte d'Ivoire), une suite de tests automatisés a été mise en place pour garantir la qualité du code après l'ajout de fonctionnalités importantes : gestion multi-magasins, utilisateurs multi-magasins, dédiée aux magasins.

Fonctionnalités couvertes

- Authentification JWT avec sessions révocables en base
- Gestion multi-magasins (jonction table utilisateurs_magasins)
- CRUD utilisateurs avec assignation à plusieurs magasins
- CRUD magasins (liste, détail, création, modification)
- Hook React useMagasin() — logique mono / multi / admin
- Formatters FCFA, dates, badges (formatters.ts)
- Trigger PostgreSQL trg_paiement_vente (recalcul automatique)

Stack de test

Framework Vitest 4.x (API + Web)

Tests API Supertest (requêtes HTTP contre l'app Express)

Tests Web React Testing Library + renderHook

Base de données PostgreSQL 16 — base dédiée telepro_test

Isolation TRUNCATE des tables opérationnelles entre chaque test

- apps/api/vitest.config.ts !' Config intégration (DB, maxWorkers: 1)
- apps/api/vitest.unit.config.ts !' Config unitaire (sans DB)
- apps/api/src/test/globalSetup.ts !' Drop/recréation telepro_test + migrations
- apps/api/src/test/setup.ts !' TRUNCATE avant chaque test
- apps/api/src/test/fixtures.ts !' makeToken(), authHeader(), request
- apps/api/src/__tests__/unit/ !' Tests helpers + scopeMagasin
- apps/api/src/__tests__/integration/!' Auth, Admin, Magasins, Triggers
- apps/web/vitest.config.ts !' Config jsdom + React Testing Library
- apps/web/src/test/setup.ts !' jest-dom + cleanup localStorage
- apps/web/src/__tests__/ !' formatters.test.ts, useMagasin.test.ts

Isolation de la base de test

À chaque lancement, globalSetup.ts recrée la base telepro_test depuis zéro (DATABASE), puis applique toutes les migrations (001!'016) instruction par ins un schéma propre et évite les conflits entre runs successifs.

Exécution séquentielle obligatoire

Les tests d'intégration partagent une base de données commune. Pour éviter TRUNCATE d'un worker qui supprime des données utilisées par un autre), la maxWorkers: 1 — les fichiers de test sont exécutés en série.

Commandes npm

```
npm run test:unit  (api) Unitaires
uniquement — sans connexion DB
npm run test:int   (api) Intégration —
recréation DB automatique
npm test          (api) Unit + intégration en
séquence
npm test          (web) Tous les tests React
(jsdom)
```

Fichier de test	Type	Tests	Statut
unit/helpers.test.ts	Unitaire	8 tests	' PASS
unit/scopeMagasin.test.ts	Unitaire	9 tests	' PASS
integration/auth.test.ts	Intégration	10 tests	' PASS
integration/admin.test.ts	Intégration	11 tests	' PASS
integration/magasins.test.ts	Intégration	11 tests	' PASS
integration/triggers.test.ts	Intégration	1 test	' PASS

Tests Web

Fichier de test	Type	Tests	Statut
__tests__/formatters.test.ts	Unitaire	19 tests	' PASS
__tests__/useMagasin.test.ts	Unitaire	19 tests	' PASS

Cas testés — highlights

helpers.test.ts

- ok() renvoie
{ success: true,
data }
- fail() renvoie
{ success: false,
error }
- wrap()
propage les erreurs
async vers next()

scopeMagasin.test.ts

- Mono-
magasin : query
param ignoré,
retourne le seul
magasin
- Admin (ids
vide) : query param
libre
- Multi-
magasin : query
param validé dans
le périmètre

auth.test.ts

- Login retourne
token +
refreshToken +
magasin_ids[]
- Blocage sur
mauvais mot de
passe (401)
- /me retourne
le profil avec
magasin_ids en
tableau JS
- Refresh
échange un token
valide

admin.test.ts

- CRUD

utilisateurs avec
vérification admin
(403 sinon)

- Création avec
magasin_ids[]
multiples via
junction table
- Mise à jour
des magasins
assignés

magasins.test.ts

- Liste avec
nb_utilisateurs et
valeur_stock
- Détail avec
utilisateurs et stocks
- CRUD protégé
admin (403 pour
commercial)

triggers.test.ts

- INSERT
paiement !'
montant_paye/
solde_restant mis à
jour
- Statut !' partiel
puis paye
automatiquement

1. Migrations multi-instructions : rollback silencieux

Problème : node-postgres exécute un fichier SQL multi-instructions dans une transaction implicite. Si une instruction échoue (ex. ALTER TABLE echecs_manuelles dans 013 avant que la table existe), toutes les instructions précédentes du fichier sont annulées.

Solution : globalSetup découpe chaque fichier en instructions individuelles (splitStatements()) et les exécute une à une avec / catch. Ainsi, un échec isolé ne compromet pas le reste.

2. CREATE OR REPLACE VIEW ne peut pas changer les colonnes

Problème : Migrations 007 et 014 tentaient de remplacer des vues existantes en modifiant leur ordre ou type de colonnes — PostgreSQL le refuse avec "cannot change name/type of view column".

Solution : Ajout de DROP VIEW IF EXISTS... CASCADE avant chaque CREATE VIEW dans les migrations concernées.

3. Arrays PostgreSQL retournés comme chaînes

Problème : La vue v_utilisateurs_magasins utilisait COALESCE(..., '{}') avec un littéral texte non typé. pg retournait la colonne en tant que text et non integer[], donc magasin_ids arrivait au frontend comme la chaîne "{1,2}" plutôt qu'un tableau [1, 2].

Solution : Cast explicite : COALESCE(array_agg(...), '{} '::int[]). Le driver pg parse alors correctement l'integer[] en tableau JavaScript.

4. Sessions TRUNCATE entre beforeAll et les tests

Problème : Le beforeAll des fichiers d'intégration créait des tokens (sessions en DB), puis le beforeEach global tronquait la table sessions, invalidant les tokens avant même que les tests ne s'exécutent (!' 401 systématique).

Solution : Suppression de sessions de la liste TRUNCATE. Chaque token a un JTI unique ; les anciennes sessions n'interfèrent pas avec les nouvelles.

5. Tests d'intégration parallèles (TRUNCATE croisé)

Problème : Par défaut, vitest exécute les fichiers de test en parallèle sur plusieurs workers. Le beforeEach d'un worker tronquait les données en cours d'utilisation par un autre, causant des failures aléatoires (SELECT retourne 0 lignes après INSERT).

Solution : Ajout de maxWorkers: 1 dans vitest.config.ts : les fichiers s'exécutent en série dans le même processus, éliminant les interférences.

6. pool.end() ferme le pool partagé entre fichiers

Problème : setup.ts appelait pool.end() dans afterAll. Comme le pool est un singleton, le premier fichier terminé fermait la connexion pour tous les suivants.

Solution : Suppression de pool.end() — le pool se ferme naturellement à la fin du processus vitest.

7. app.listen() déclenchée lors de l'import par supertest

Problème : src/index.ts appelait app.listen(3001) au niveau module. Supertest importe l'app, ce qui déclenchait listen(), causant EADDRINUSE si le serveur était déjà démarré.

Solution : Condition if (process.env.NODE_ENV !== 'test') autour de app.listen().

Résultats finaux

Tests web (React / jsdom) 88 / 88

Tests API unitaires 17 / 17

Tests API intégration 33 / 33

Total 88 / 88 — 100 % de réussite

Couverture fonctionnelle atteinte

'Authentification JWT Token Gestion multi-magasins (junction table)
'CRUD admin utilisateurs + magasins
'Hook useMagasin() (mono / multi-admin)
'Formatters monétaires et dates
'Triggerer le paiement
'Isolation

DB
entr
e te
sts (
TR
UN
CAT
E)
'Exé
cuti
on d
éter
mini
ste (
pas
de fl
akin
ess)

Recommandations pour la suite

- Ajouter des tests pour les routes /ventes, /clients, /stock (endpoints principaux)
- Tester les notifications SMS/WhatsApp avec un mock Twilio
- Intégrer les tests dans la CI/CD (GitHub Actions ou Docker Compose test stage)
- Ajouter des tests de charge légers sur les endpoints les plus critiques
- Couvrir les cas d'erreur réseau et timeout DB dans les routes Express